

Unit II :

Basic of c#

2.1. Introduction

1. C#.NET is one of the Microsoft Programming Languages to work with the .NET Framework, .NET Core, or .NET to develop different kinds of applications such as Web, Console, Windows, Restful Services, etc.
2. It is the most powerful programming language among all programming languages available in the .NET framework because it contains all the features of C++, VB.NET, and JAVA, and also has some additional features. As we progress in this course, you will come to know the additional features.
3. C#.NET is a completely Object-Oriented Programming Language. It means it supports all 4 OOPs Principles i.e. Abstraction, Encapsulation, Inheritance, and Polymorphism.

Console Application

1. A console application is an application that can be run in the command prompt. For any beginner on .NET or anyone who wants to learn C# Language or anyone who wants to become an expert in C# Language, building a console application is ideally the first step to learning the C# Language.
2. The Console Applications contain a similar user interface to the Operating systems like MS-DOS, UNIX, etc.
3. The Console Application is known as the CUI application because in this application we completely work with the CUI environment.
4. These applications are similar to C or C++ applications.
5. Console applications do not provide any GUI facilities like the Mouse Pointer, Colors, Buttons, Menu Bars, etc.

Basic Structure of the C# Program

Now, let's understand the Basic Structure of the C# Program using a Console Application.

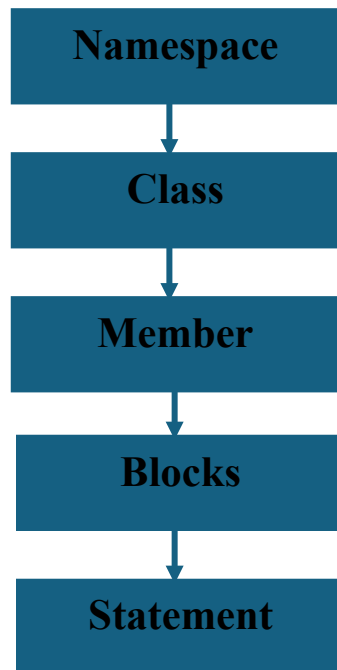
```
//Include classlibraries/namespace
using System;
//declare namespace
namespace namespacename
{
    //declaration of Class
    class ClassName
```

```

{
    //Data Member
    Int SomeMember1;
    float SomeMember2;
    //Function/Methods/Block
    static void SomeBlock()
    {
        //Statement
        Console.WriteLine("Hello World");
    }
}

```

The above process is shown in the below diagram.



Note: C#.NET is a Case-Sensitive Language and Every Statement in C# should end with a Semicolon.

Importing Namespace Section:

This section contains importing statements that are used to import the BCL (Base Class Libraries) as well as user-defined namespaces if required. This is similar to the included statements in the C programming language. Suppose you want to use some classes and interfaces in your code, then you have to include the namespace(s) from where these classes and interfaces are defined. For example, if you are going to use the Console class in your code, then you have to include the System namespace as the Console class belongs to the System namespace.

Syntax: `using NamespaceName;`

Example: `using System;`

Note: A namespace is a container that contains a group of related classes and interfaces, as well as, a namespace can also contain other namespaces.

Namespace Declaration Section:

Here a user-defined namespace is declared. In .NET applications, all classes and interfaces or any type related to the project should be declared inside some namespace. Generally, we put all the related classes under one namespace and in a project, we can create multiple namespaces.

Syntax: namespace NamespaceName {}

Example: namespace MyFirstProject {}

Generally, the namespace name will be the same as the project name but it is not mandatory, you can give any user-defined name to the namespace.

Namespace Declaration Section:

Here a user-defined namespace is declared. In .NET applications, all classes and interfaces or any type related to the project should be declared inside some namespace. Generally, we put all the related classes under one namespace and in a project, we can create multiple namespaces.

Syntax: namespace NamespaceName {}

Example: namespace MyFirstProject {}

Generally, the namespace name will be the same as the project name but it is not mandatory, you can give any user-defined name to the namespace.

Class Declaration Section:

For every Desktop Application in .NET, we need a start-up class file. For example, for every .NET Desktop Application like Console and Windows, there should be a Start-Up class that should have the Main method from where the program execution is going to start. When we create a Console Application using Visual Studio, by default, Visual Studio will Create the Start-Up class file with the name Program.cs which will have a class with the name Program that contains the Main method. A start-up class is nothing but a class that contains a Main() method from which the program execution is going to start.

Syntax:

class ClassName

{
}

Example:

class Program

{

}

Note: You can change this default behavior. You can create your own class and you can also make that class as the Start-Up Class by including the Main method. In our upcoming articles, we will discuss this in detail.

Main() Method Section:

The main() method is the entry point or starting point of the application to start its execution. When the application starts executing, the main method will be the first block of the application to be executed. The Main method contains the main logic of the application.

What is using?

Using is a keyword. Using this keyword, we can refer to .NET BCL in C# Applications i.e. including the BCL namespaces as well as we can also include user-defined namespaces that we will discuss as we progress in this course. Apart from importing the namespace, other uses of using statements are there, which we will also discuss as progress in this course. For now, it is enough.

Note: In .NET the base class libraries are divided into a collection of namespaces. Each namespace contains a set of predefined classes and sub-namespaces. The namespace contains another namespace called sub-namespaces.

Methods and Properties of Console Class in C#

Properties of Console Class in C#:

There are many properties available in the Console class. Some of them are as follows:

1. **Title:** It gets or sets the title to display in the console title bar. It returns the string to be displayed in the title bar of the console. The maximum length of the title string is 24500 characters.
2. **BackgroundColor:** It gets or sets the background color of the console. It returns a value that specifies the background color of the console; that is, the color that appears behind each character. The default is black.
3. **ForegroundColor:** It gets or sets the foreground color of the console. It returns a ConsoleColor that specifies the foreground color of the console; that is, the color of each character that is displayed. The default is gray.
4. **CursorSize:** It gets or sets the height of the cursor within a character cell. It returns the size of the cursor expressed as a percentage of the height of a character cell. The property value ranges from 1 to 100.

There are many methods available in the Console class. Some of them are as follows:

1. **Clear():** It is used to clear the console buffer and corresponding console window of display information. In simple words, it is used to clear the screen.

2. **Beep():** This method plays the sound of a beep through the console speaker. That means it plays a beep sound using a PC speaker at runtime.
3. **ResetColor():** This method is used to set the foreground and background console colors to their defaults.
4. **Write("string"):** This method is used to write the specified string value to the standard output stream.
5. **WriteLine("string"):** This method is used to write the specified string value, followed by the current line terminator, to the standard output stream. That means this method same as the write method but automatically moves the cursor to the next line after printing the message.
6. **Write(variable):** This method is used to write the value of the given variable to the standard output stream.
7. **WriteLine(variable):** This method is used to write the value of the given variable to the standard output stream along with moving the cursor to the next line after printing the value of the variable.
8. **Read():** This method read a single character from the keyboard and returns its ASCII value. The Datatype should be int as it returns the ASCII value.
9. **ReadLine():** This method reads a string value from the keyboard and returns the entered value only. As it returns the entered string value so the DataType is going to be a string.
10. **ReadKey():** This method reads a single character from the keyboard and returns that character information like what key has been entered, and what its corresponding ASCII value is. The Datatype should be ConsoleKeyInfo which contains the entered key information

Example to show the use of the Write and WriteLine method in C#:

```
//Program to show the use of WriteLine and Write Method
//First Import the System namespace as the
//Console class belongs to System namespace
using System;
namespace MyFirstProject
{
    internal class Program
    {
        static void Main(string[] args)
        {
            //We can access WriteLine and Write method using class name
            //as these methods are static

            //WriteLine Method Print the value and move the cursor to the next
line
```

```

    Console.WriteLine("Hello");
    //Write Method Print the value and stay in the same line
    Console.Write("HI ");
    //Write Method Print the value and stay in the same line
    Console.Write("Bye ");
    //WriteLine Method Print the value and move the cursor to the next
line
    Console.WriteLine("Welcome");
    //Write Method Print the value and stay in the same line
    Console.Write("C#.NET ");
    Console.ReadKey();
}
}
}

```

Example to show how to print the value of a variable in C#.

```

//Program to show how to print the value of a variable
using System;
namespace MyFirstProject
{
    internal class Program
    {
        static void Main(string[] args)
        {
            string name = "Pranaya";
            Console.WriteLine(name);
            Console.WriteLine("Hello " + name);
            Console.WriteLine($"Hello {name}");
            Console.ReadKey();
        }
    }
}

```

Reading Value from the user in C#:

```

//Program to show how to read value at runtime
using System;
namespace MyFirstProject
{
    internal class Program

```

```

{
    static void Main(string[] args)
    {
        //Giving one message to the user to enter his name
        Console.WriteLine("Enter Your Name");

        //ReadLine method reads a string value from the keyboard
        string name = Console.ReadLine();

        //Printing the entered string in the console
        Console.WriteLine($"Hello {name}");
        Console.ReadKey();
    }
}

```

Read Integer Numbers in C# from the keyword

```

//Program to show how to read integer values
using System;
namespace MyFirstProject
{
    internal class Program
    {
        static void Main(string[] args)
        {
            Console.WriteLine("Eneter two Numbers:");

            //Converting string to Integer
            int Number1 = Convert.ToInt32(Console.ReadLine());

            //Converting string to Integer
            int Number2 = Convert.ToInt32(Console.ReadLine());

            int Result = Number1 + Number2;
            Console.WriteLine($"The Sum is: {Result}");
            Console.WriteLine($"The Sum is: {Number1 + Number2}");
            Console.ReadKey();
        }
    }
}

```

Example to understand Console Class Properties:

```

//Program to show the use of Console Class Properties and Beep Method

```

```
using System;
namespace MyFirstProject
{
internal class Program
{
static void Main(string[] args)
{
    Console.BackgroundColor = ConsoleColor.Blue;
    Console.ForegroundColor = ConsoleColor.White;
    Console.Title = "Understanding Console Class";
    Console.WriteLine("BackgroundColor: Blue");
    Console.WriteLine("ForegroundColor: White");
    Console.WriteLine("Title: Understanding Console Class");
    //It will make Beep Sound
    Console.Beep(0);
    Console.ReadKey(0);
}
}
}
```